

Most developers try to learn Design Patterns immediately, but many Design Patterns exist because SOLID principles solve common design problems first.

A good learning path is:

1. Clean Code
2. SOLID Principle
3. Design Pattern
4. Architecture Pattern
5. System Design

If we don't understand SOLID, they usually misuse Design Patterns.

What is SOLID?

SOLID is a set of 5 object-oriented design principles introduced by Robert C. Martin.

The goal is to create code that is:

- Easy to understand
- Easy to change
- Easy to test
- Flexible
- Maintainable

S = Single Responsibility Principle (SRP)

Definition

A class should have only one reason to change.

Many developers misunderstand this as:

One class should do only one thing.

Not exactly.

The correct meaning is:

A class should be responsible for one business responsibility.

Bad Example

```
@Service
public class UserService {

    public void register(User user) {

        // validate
        validate(user);

        // save
        userRepository.save(user);

        // send email
        emailService.sendWelcomeEmail(user);

        // generate report
        reportService.generate(user);
    }
}
```

Problems:

- Validation changes
- Database changes
- Email changes
- Report changes

All affect this class.

Better Design

```
@Component
public class UserValidator {
```

```
}
```

```
@Component  
public class UserWriter {  
}
```

```
@Component  
public class WelcomeEmailSender {  
}
```

```
@Service  
public class RegisterUserUseCase {  
}
```

Each class has one responsibility.

Real Example From Product Catalog Project

Good examples:

CreateProductUseCase
UpdateProductUseCase
DeleteProductUseCase

ProductImageLoader
ProductVariantLoader

ProductResponseAssembler

These follow SRP much better than one giant ProductService.

O = Open Closed Principle (OCP)

Definition

Software entities should be:

- Open for extension
- Closed for modification

Meaning:

You should add new behavior without changing existing code.

Bad Example

```
public BigDecimal calculateDiscount(
    String customerType,
    BigDecimal amount) {

    if ("SILVER".equals(customerType)) {
        return amount.multiply(BigDecimal.valueOf(0.05));
    }

    if ("GOLD".equals(customerType)) {
        return amount.multiply(BigDecimal.valueOf(0.10));
    }

    if ("PLATINUM".equals(customerType)) {
        return amount.multiply(BigDecimal.valueOf(0.15));
    }

    return BigDecimal.ZERO;
}
```

Whenever a new membership appears:

DIAMOND

You modify the class.

Violation of OCP.

Better Design

```
public interface DiscountPolicy {  
  
    BigDecimal calculate(BigDecimal amount);  
}
```

```
@Component  
public class SilverDiscountPolicy  
    implements DiscountPolicy {  
}
```

```
@Component  
public class GoldDiscountPolicy  
    implements DiscountPolicy {  
}
```

```
@Component  
public class PlatinumDiscountPolicy  
    implements DiscountPolicy {  
}
```

Now adding Diamond:

```
@Component  
public class DiamondDiscountPolicy  
    implements DiscountPolicy {  
}
```

No existing code changes.

Design Patterns Related

- Strategy Pattern
- Factory Pattern
- Template Method Pattern

exist largely because of OCP.

L = Liskov Substitution Principle (LSP)

Definition

A subclass should be able to replace its parent without breaking behavior.

Classic Bad Example

```
class Bird {  
    void fly() {  
    }  
}
```

```
class Penguin extends Bird {  
}
```

Problem:

Penguins cannot fly.

Later:

```
Bird bird = new Penguin();  
bird.fly();
```

System becomes incorrect.

Better Design

```
interface Bird {  
}
```

```
interface FlyingBird extends Bird {  
  
    void fly();  
}
```

```
class Eagle implements FlyingBird {  
}
```

```
class Penguin implements Bird {  
}
```

Now behavior is correct.

Enterprise Example

Bad:

PaymentService

CreditCardPaymentService

CashPaymentService

If CashPaymentService throws:

```
throw new UnsupportedOperationException();
```

for some methods required by parent,

LSP is violated.

I = Interface Segregation Principle (ISP)

Definition

Clients should not depend on methods they don't use.

Bad Example

```
public interface Worker {  
  
    void work();  
  
    void eat();  
  
    void sleep();  
  
    void drive();  
}
```

Developer

```
public class Developer implements Worker {  
}
```

Why must a Developer implement:

drive()

?

Makes no sense.

Better Design

```
interface Worker {  
    void work();  
}
```

```
interface Driver {  
    void drive();  
}
```

```
interface Eater {  
    void eat();  
}
```

Small focused interfaces.

Enterprise Example

Bad:

ProductService

with 50 methods.

```
create()  
update()  
delete()
```

```
uploadImage()  
search()  
export()  
import()  
...
```

Huge interface.

Better:

ProductCommandService

ProductQueryService

ProductImportService

ProductExportService

D = Dependency Inversion Principle (DIP)

This is probably the most important principle in Spring Boot.

Definition

Depend on abstractions, not concrete implementations.

Bad Example

```
@Service  
public class OrderService {
```

```
private final MySqlOrderRepository repository =  
    new MySqlOrderRepository();  
}
```

OrderService is tightly coupled.

Better Design

```
public interface OrderRepository {  
  
    Mono<Order> save(Order order);  
}
```

Implementation:

```
@Repository  
public class PostgresOrderRepository  
    implements OrderRepository {  
}
```

Use abstraction:

```
@Service  
@RequiredArgsConstructor  
public class OrderService {  
  
    private final OrderRepository repository;  
}
```

Now OrderService doesn't care whether storage is:

- PostgreSQL
- MongoDB
- Redis
- Mock Repository

Why Spring Exists

Spring Dependency Injection is largely an implementation of DIP.

```
@RequiredArgsConstructor
public class CreateProductUseCase {

    private final ProductRepository repository;
}
```

This is DIP.